Blog of @ymmt2005

2020-04-14

An example of using dynamic client of k8s.io/client-go

Kubernetes Programming Go

In this article, I describe how to write a Kubernetes client in Go using the *dynamic* client in k8s.io/client-go package. During the course, you can learn the following things:

- The difference between typed clients and the dynamic client.
- Reading YAML manifests into `unstructured.Unstructured`.
- Discovering the REST API endpoint for a Group-Version-Kind.
- Creating and updating resources using Server Side Apply.

You should have a basic knowledge of Kubernetes and Go programming. The examples in this article depend on k8s.io/client-go@v0.18.1.

- Background: Server Side Apply
- Kubernetes API Basics

    - Mapping between GVK and GVR
- Go client libraries
    - Typed clients
    - Dynamic client
- Using the dynamic client to implement SSA

Background: Server Side Apply

Recently, I wrote a program that applies Kubernetes resources using Server Side Apply. Server Side Apply, or **SSA**, is a new way to create or update resources in Kubernetes API server added as a beta feature to Kubernetes 1.16.

One of the advantage of SSA is that it introduces better patching strategy than Strategic Merge Patch. For example, if a Service has two ports sharing the same port number but with different protocol, Strategic Merge Patch could not identify which port should be updated because it uses `port` as the key.

```
apiVersion: v1
kind: Service
metadata:
  name: mydns
spec:
  selector:
    app: mydns
  ports:
    - protocol: TCP
      port: 53
    - protocol: UDP
      port: 53
```

On the other hand, SSA can identify the two ports because it can define a tuple of fields as the merge key. The two `+listMapKey=` comments in the

ServiceSpec defines the key for SSA.

```
type ServiceSpec struct {
    // The list of ports that are exposed by this service.
    // More info: https://kubernetes.io/docs/concepts/services-network
    // +patchMergeKey=port
    // +patchStrategy=merge
    // +listType=map
    // +listMapKey=port
    // +listMapKey=protocol
    Ports []ServicePort `json:"ports,omitempty" patchStrategy:"merge"
}
```

SSA manages *the owner* of each field in the resource and allows only the owner to update the field. The field owner is recorded when a client creates or updates (in non-SSA sense) fields of a resource. Therefore, it is mandatory for SSA to send a partial object that **includes only the fields that the client wants to manage**.

When developing a Kubernetes client in Go, the most commonly used package is `k8s.io/client-go`. The package contains two kinds of client library, namely, *dynamic* and *typed*.

For SSA, you need to use the *dynamic* client because you cannot send a partial object using *typed* clients.

Kubernetes API Basics

Ref: Kubernetes API Overview

Most of the Kubernetes API are *resources* stored in the API server. Each resource type has a concrete representation in JSON or equivalent YAML called *kind*.

The resources and kinds are grouped and versioned. For example, `deployments` resource is in `apps` group and versioned as `v1`. The kind of `deployments` is Deployment.

So, a concrete representation of a `deployments` resource in YAML looks like:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-app
  namespace: default
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
```

A resource type can be uniquely identified by a tuple of Group, Version, and Resource (**GVR** for short). Similarly, a kind can be uniquely identified by a tuple of Group, Version, and Kind (**GVK** for short).

Mapping between GVK and GVR

GVR is used to compose a REST API request. For example, a REST API request for `apps`, `v1`, `deployments` looks like:

```
GET /apis/apps/v1/namespaces/{namespace}/deployments/{name}
```

By reading JSON or YAML of a resource, you can obtain the GVK for it. If there is a mapping between GVK and GVR, you can send a REST API request of a resource read from YAML. Such a mapping is called REST mapping.

The Kubernetes API server provides a **discovery API** to find all available REST mappings. You can see the human readable result of the discovery API through `kubectl api-resources` output as follows:

```
$ kubectl api-resources --api-group=apps
NAME                  SHORTNAMES   APIGROUP   NAMESPACED   KIND
controllerrevisions                apps       true         ControllerR
daemonsets            ds           apps       true         DaemonSet
deployments           deploy       apps       true         Deployment
replicasets           rs           apps       true         ReplicaSet
statefulsets          sts          apps       true         StatefulSet
```

Go client libraries

The most commonly used Go client libraries are in k8s.io/client-go package. The package depends k8s.io/api that is a collection of structs for kinds, and k8s.io/apimachinery that implements GVK, GVR, and other utilities.

Typed clients

This kind of clients uses Go structs to represent a kind. You can edit resources in a type-safe way with them. Moreover, they can automatically find the REST mappings to send API requests. Following is an example of using a typed client to create a Deployment resource.

```
import (
    "context"
```

```
    appsv1 "k8s.io/api/apps/v1"
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/rest"
)

// Create a Deployment in a namespace ns.
func createDeployment(ctx context.Context, config *rest.Config, ns str
    clientset, err := kubernetes.NewForConfig(config)
    if err != nil {
        return err
    }

    deployment := &appsv1.Deployment{}
    deployment.Name = "example"
    // edit deployment spec

    client := clientset.AppsV1().Deployments(ns)
    _, err = client.Create(ctx, deployment, metav1.CreateOptions{})
    return err
}
```

In almost all cases, you should prefer typed clients to the dynamic client *except in the case of SSA*. Specifically, you must avoid using Go structs for kinds.

As SSA patches need to be sent in JSON or YAML, marshaling Go struct would add unintended fields. Let's see how `DaemonSet` struct is marshaled:

```
package main

import (
    "encoding/json"
    "os"

    appsv1 "k8s.io/api/apps/v1"
)

func main() {
```

```
    ds := &appsv1.DaemonSet{}
    ds.Name = "example"
    // edit deployment spec

    enc := json.NewEncoder(os.Stdout)
    enc.SetIndent("", "    ")
    enc.Encode(ds)
}
```

Running the above example will show the following JSON:

```
{
    "metadata": {
        "name": "example",
        "creationTimestamp": null
    },
    "spec": {
        "selector": null,
        "template": {
            "metadata": {
                "creationTimestamp": null
            },
            "spec": {
                "containers": null
            }
        },
        "updateStrategy": {}
    },
    "status": {
        "currentNumberScheduled": 0,
        "numberMisscheduled": 0,
        "desiredNumberScheduled": 0,
        "numberReady": 0
    }
}
```

Alas, there are a LOT of fields! Applying this JSON with SSA would claim the ownership of these unintended fields.

Dynamic client

Enter the dynamic client, k8s.io/client-go/dynamic. The dynamic client does not use Go types defined in k8s.io/api. Instead, it uses `unstructured.Unstructured`.

Think of `unstructured.Unstructured` as a straightforward representation of YAML/JSON object in Go. It provides utilities to handle common attributes of resources such as `metadata.namespace`. Encoding `unstructured.Unstructured` back into JSON/YAML does not add extra fields.

The following example demonstrates how to read YAML manifest into `unstructured.Unstructured` and encode it back into JSON.

```
package main

import (
    "encoding/json"
    "fmt"
    "os"

    "k8s.io/apimachinery/pkg/apis/meta/v1/unstructured"
    "k8s.io/apimachinery/pkg/runtime/serializer/yaml"
)

const dsManifest = `
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: example
  namespace: default
spec:
  selector:
    matchLabels:
      name: nginx-ds
  template:
    metadata:
      labels:
        name: nginx-ds
```

```
    spec:
      containers:
      - name: nginx
        image: nginx:latest
`

func main() {
    obj := &unstructured.Unstructured{}

    // decode YAML into unstructured.Unstructured
    dec := yaml.NewDecodingSerializer(unstructured.UnstructuredJSONSch
    _, gvk, err := dec.Decode([]byte(dsManifest), nil, obj)

    // Get the common metadata, and show GVK
    fmt.Println(obj.GetName(), gvk.String())

    // encode back to JSON
    enc := json.NewEncoder(os.Stdout)
    enc.SetIndent("", "    ")
    enc.Encode(obj)
}
```

The output should be:

```
example apps/v1, Kind=DaemonSet
{
    "apiVersion": "apps/v1",
    "kind": "DaemonSet",
    "metadata": {
        "name": "example",
        "namespace": "default"
    },
    "spec": {
        "selector": {
            "matchLabels": {
                "name": "nginx-ds"
            }
        },
        "template": {
            "metadata": {
```

```
                "labels": {
                    "name": "nginx-ds"
                }
            },
            "spec": {
                "containers": [
                    {
                        "image": "nginx:latest",
                        "name": "nginx"
                    }
                ]
            }
        }
    }
}
```

Unlike typed clients, you need to provide GVR to the dynamic client. As mentioned before, you can find the corresponding GVR for a GVK by querying API server. This can be written in Go as follows:

```
import (
    "k8s.io/apimachinery/pkg/api/meta"
    "k8s.io/apimachinery/pkg/runtime/schema"
    "k8s.io/client-go/discovery"
    "k8s.io/client-go/discovery/cached/memory"
    "k8s.io/client-go/rest"
    "k8s.io/client-go/restmapper"
)

// find the corresponding GVR (available in *meta.RESTMapping) for gvk
func findGVR(gvk *schema.GroupVersionKind, cfg *rest.Config) (*meta.RE

    // DiscoveryClient queries API server about the resources
    dc, err := discovery.NewDiscoveryClientForConfig(cfg)
    if err != nil {
        return nil, err
    }
    mapper := restmapper.NewDeferredDiscoveryRESTMapper(memory.NewMemC
```

```
    return mapper.RESTMapping(gvk.GroupKind(), gvk.Version)
}
```

Using the dynamic client to implement SSA

Now that we are ready, let's see how to implement SSA using the dynamic client.

First, we need to prepare `*rest.Config`. If the program works in a Kubernetes cluster, it is as easy as just calling `rest.InClusterConfig`.

The rest of the work is shown below with plenty of annotations.

```
import (
    "context"
    "encoding/json"

    "k8s.io/apimachinery/pkg/api/meta"
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
    "k8s.io/apimachinery/pkg/apis/meta/v1/unstructured"
    "k8s.io/apimachinery/pkg/runtime/serializer/yaml"
    "k8s.io/apimachinery/pkg/types"
    "k8s.io/client-go/discovery"
    "k8s.io/client-go/discovery/cached/memory"
    "k8s.io/client-go/dynamic"
    "k8s.io/client-go/rest"
    "k8s.io/client-go/restmapper"
)

const deploymentYAML = `
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: default
spec:
  selector:
```

```
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
`

var decUnstructured = yaml.NewDecodingSerializer(unstructured.Unstruct

func doSSA(ctx context.Context, cfg *rest.Config) error {

    // 1. Prepare a RESTMapper to find GVR
    dc, err := discovery.NewDiscoveryClientForConfig(cfg)
    if err != nil {
        return err
    }
    mapper := restmapper.NewDeferredDiscoveryRESTMapper(memory.NewMemC

    // 2. Prepare the dynamic client
    dyn, err := dynamic.NewForConfig(cfg)
    if err != nil {
        return err
    }

    // 3. Decode YAML manifest into unstructured.Unstructured
    obj := &unstructured.Unstructured{}
    _, gvk, err := decUnstructured.Decode([]byte(deploymentYAML), nil,
    if err != nil {
        return err
    }

    // 4. Find GVR
    mapping, err := mapper.RESTMapping(gvk.GroupKind(), gvk.Version)
    if err != nil {
        return err
    }

    // 5. Obtain REST interface for the GVR
    var dr dynamic.ResourceInterface
```

```
    if mapping.Scope.Name() == meta.RESTScopeNameNamespace {
        // namespaced resources should specify the namespace
        dr = dyn.Resource(mapping.Resource).Namespace(obj.GetNamespace
    } else {
        // for cluster-wide resources
        dr = dyn.Resource(mapping.Resource)
    }

    // 6. Marshal object into JSON
    data, err := json.Marshal(obj)
    if err != nil {
        return err
    }

    // 7. Create or Update the object with SSA
    //     types.ApplyPatchType indicates SSA.
    //     FieldManager specifies the field owner ID.
    _, err = dr.Patch(ctx, obj.GetName(), types.ApplyPatchType, data,
        FieldManager: "sample-controller",
    })

    return err
}
```

That's it!

ymmt (id:ymmt2005) 2 years ago

0
Tweet
シェアする



Sponsor Yamamoto, Hirotaka on GitHub Sponsors

ymmt2005 is a long-time open-source contributor and blog writer in Japa
open-source software including MOCO, Coil v2, transocks & usocksd, and

anujjangra25119@gmail.com

Awesome blog. Very helpful

2 years ago

Naza

Thanks a lot for your tutorial! It really saved me the hours!!!

2 years ago

Mark Miller

Hi, I'm getting a "server could not find the resource" error doing the PATCH. Does the FieldManager option need to be something specific to my code/system?

2 years ago

Read more

Write a comment



« Use go-grpc-middleware with kubebuilder

Writing and testing Kubernetes webhooks… »

Profile

https://github.com/ymmt2005

Subscribe 6

About this blog

Search

Search entries

Recent Entries

Caching Unstructured Objects using controller-runtime

A Rootless Squid Container Image

47 Things To Become a Kubernetes Expert

Use go-grpc-middleware with kubebuilder

An example of using dynamic client of k8s.io/client-go

Archive

- ▸ 2021 (3)
- ▼ 2020 (2)
  - 2020 / 8 (1)
  - 2020 / 4 (1)
- ▸ 2019 (1)
- ▸ 2016 (5)